Project 5: Tactical Strategy

11/24/2025

50 Points Possible

Attempt 1

In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

Details

Introduction

This is it! Your excellent work on various MMO systems has resulted in you finally getting to work on core gameplay logic. You've been given technical direction over an entirely new gameplay system: tactical turn-based strategy.

The design team feels that the core gameplay needs to be broken up with a different kind of interaction, so instead of combat or competitive basket weaving, units will be engaging in very serious philosophical debates. Each player will control a group of debaters, and on their turn, will order one debater to engage another debator in philosophical battle.

Your job as engineer on this project is to handle path-finding. Specifically, your input will be a grid-based map. Your goal is to compute the cost to move the selected character adjacent to the selected target character.

Input / output

The input will be a grid of space-separated single characters sent to standard input. Each character represents a different kind of terrain. Each kind of terrain has a different movement cost. Movement cost is payed upon entering a tile. Terrain types all have different costs and symbols, which are as follows:

Road: '.', cost: 1
Grass: ',', cost: 2
Rocks: 'o', cost: 3
Walls: '=', cost: 50
Character: '0', '1', '2', or '3'
('0' represents the controlled character, '1' represents a friendly character, '2' represents the target character, and '3' represents a non-target enemy character)
(two humans cannot be in the same tile, so no cost is listed)
The target character is not allowed to move through any character.

Additionally, if a player moves diagonally, it counts as 1.5x the number of movement points as the tile type indicates (so moving diagonally into rocks costs 4.5 points).

The input will consist of a grid of tiles like this:

```
, , , , , , , , .
. . . . . . . . .
. . . . = = = = ,
. 1 . . . 0 . = .
. = = = = 1 . = ,
. . . . = = = = ,
, . . . , , , , ,
, . . . , , , , ,
, 2 3 3 , , , , ,
```

You will read in the input and determine the shortest path between the character '0' and the character '2'. The final square must be the closest adjacent square to the 2 (not the 2 itself, two characters may not occupy the same tile).

The output will be the same tile map, but with the shortest path drawn in asterisks ('*'). Here is the output for the input above.

```
, , , , , , , , .
. . . . . . . . .
. * . . = = = = ,
* 1 * * * 0 . = .
```

```
* = = = = 1 . = ,
* . . . = = = = ,
, * . . , , , , ,
, * . . , , , , ,
, 2 3 3 , , , , ,
```

Additionally, print the total cost afterward.
Total cost: 10.5

This is very important, because there may be more than one ideal path, but everyone should get the same total cost.

So the complete output would be

```
, , , , , , , , .
. . . . . . . . .
. * . . = = = = ,
* 1 * * * 0 . = .
* = = = = 1 . = ,
* . . . = = = = ,
, * . . , , , , ,
, * . . , , , , ,
, 2 3 3 , , , , ,
Total cost: 10.5
```

Warning: this sample input uses a 9x9 square map, but actual test inputs will be much larger, and may be rectangular. All the inputs that will be tested on are attached. Make sure your code runs on all of them.

You may assume that the target is reachable. I will not give you an input in which pathfinding completely fails.

Grading

30% Effort points
20% Used an algorithm that is faster than Dijkstra's algorithm. A* is a good choice, but you can try others. Leave a comment at the top of the submission **and on Canvas** if you use an algorithm that is faster than Dijkstra's so the TA knows to give you these points. Thank you!
50% Correct output and reasonable runtime

All your code, including loading the files, must run within 10 seconds to receive anything other than discretionary credit. Otherwise it will be a timeout, the same as if output was incorrect.

Your output should match the correct format exactly, with the correct capitalization, spacing, and punctuation. However, in this problem, there may be multiple solutions (equally short paths). If this happens, you won't be penalized. Your "total cost" line must be identical to the expected output, however.

The TA may manually grade your paths to make sure that they match up with the total cost you report. If these don't match up you are only eligible for effort points.

All tested inputs and outputs are attached.

Programming Languages

You must use C. This applies to every project in this class.

Your code will be compiled with -std=c11 -O2 -lm

How to submit

Submit your code as a single C file. If you used multiple C files or a makefile, I'm sorry to make you do this, but please find a way to cram everything into one .c file (Canvas is not really designed for programming projects, and it becomes problematic to work with if we use .zip files).

Note: you must submit on AutoLab if it is available. We will announce whether you have to submit on AutoLab and where.

Name your main file main.c.

Failure to submit correctly could result in a small penalty (5 to 10 points).

Additional Rules

- You are allowed to use standard libraries, however you must write the shortest path algorithm yourself. Feel free to use my heap code as a basis for your priority queue.
- If your code submission is corrupt, you will get a zero. Check your submissions afterward by downloading your own submission and running it.

Plagiarism

No plagiarism. It's okay to read about techniques online, but your code must be your own work. You may not work in groups. You may discuss the problem with classmates, but may not share code. Your code may be checked using an automated plagiarism checker. This is an individual assignment.

Tips

- It's possible there is more than one correct solution. The most important thing is that your total cost be correct.
- Be very careful with the cost function. Diagonals need to be handled properly.

Alignments

- MO1: Entire assignment
- MO3: Able to implement algorithm at least as fast as Dijkstra's algorithm (partial mastery) or faster (exceeds mastery) (20% + 70%)

View Rubric

Project 5 rubric

Criteria	Ratings					Points
MO1 **MO1** view longer description Threshold: 4 pts	**Exceeds Mastery** **5 pts**	**Mastery** **4 pts**	**Near Mastery** **3 pts**	**Below Mastery** **2 pts**	**No Evidence** **1 pts**	--/5 pts
MO3 **MO3** view longer description Threshold: 4 pts	**Exceeds Mastery** **5 pts**	**Mastery** **4 pts**	**Near Mastery** **3 pts**	**Below Mastery** **2 pts**	**No Evidence** **1 pts**	--/5 pts

Choose a submission type

Upload

Office 365

Choose a file to upload

File permitted: C

or

Canvas Files

I agree to the tool's **End-User License Agreement** **(https://washingtonstateuniversitysystem.turnitin.com/integrations/cpf/eula/washingtonstateuniversitysystem)**
This assignment submission is my own, original work
*

(https://wsu.instructure.com/courses/1820402/modules/items/24229247)

Assignment **(https://wsu.instructure.com/courses/1820402/modules/items/242292**